

Lab 4: Arithmetic Logic Unit

Purpose

The purpose of this experiment is to design an Arithmetic Logic Unit (ALU) which is a combinational circuit that can perform some specific operations. While designing this circuit, BASYS 3 FPGA board will be used.

Design Specifications

An ALU is a combinational circuit that can perform mathematical operations and is an essential building block for most digital devices. The ALU that will be designed in this lab will be 6-bit ALU and it will perform eight functions: addition, subtraction, logical shift left, arithmetic shift right, logical shift right, comparator (with equal, less than, and greater than outputs), rotate right, and bitwise XOR. In order to implement these eight functions, function blocks and 8-to-1 multiplexer will be used.

First, there will be two main inputs (A and B). They are 6-bit data inputs and A5, A4, A3, A2, A1, A0 assigned to R2, T1, V1, W2, R3, T2 and B5, B4, B3, B2, B1, B0 assigned to T3, V2, W13, W14, V15, W15, respectively. They will be connected to the function blocks which are performing specified operations. All function blocks will be written during the experiment process. Then, all blocks will compose the inputs of the 8-to-1 multiplexer. For multiplexer having 8 inputs, 3 selection data is needed. Therefore there will be 3 selection bits s0, s1, s2 which are assigned to V17, V16 and W16 switches, respectively.

Likewise inputs A and B, output O will also be a 6-bit data (O5, O4, O3, O2, O1, O0) who is assigned L1, P1, N3, P3, V3 and W3 LEDs, respectively.

Methodology

In the design process of 6-bit ALU, modular fashion is followed. Therefore operations are built on smaller functions.

There are 8 functions in the Arithmetic Logic Unit. These are addition, subtraction, logical shift left, arithmetic shift right, logical shift right, comparator (with equal, less than, and greater than outputs), rotate right, and bitwise XOR like mentioned. A 8-to-1 multiplexer is implemented to pass the wanted information and the control is made with selection switches on BASYS. Likewise a normal multiplexer, for each selection vector value, a function is assigned and its result is assigned to the output.

The smallest components are halfAdder and fullAdder blocks. fullAdder is composed of 2 halfAdders. In first two operations, addition and subtraction, fullAdders are used. For each bit position a fullAdder is used. Since the vectors are 6 bit vectors (using declaration 5 downto 0), 6 fullAdders are used. The difference between addition and subtraction is simply the sign change likewise it is implemented in mathematics.

In the shifting operations vectors are shifted using bitwise concatenation. In VHDL ampersand operand (&) is used for this purpose. Therefore according to left-right directions, this implementation is made. There are also slight changes in arithmetic shifts, logical shifts and rotating right. In logical shift, numbers are considered as unsigned numbers, while, signed in arithmetic shifts. According to this difference, first bit position is kept in arithmetic right change, while only shifting is present in logical shift. Additionally, rotating right is act like a loop, namely, a digit lost from a side appears on the other side.

Comparator operator evaluates the binary value of two input numbers. Since ALU is inputting and outputting 6-bit numbers, the result must be 6-bit number. It is designed as one bit position will be high in all possibilities and the other bits will be low. There are 3 possibilities which are A is greater than B (AgtB), A is equal to B (AeqB), A is less than B (AltB). The process is starting with creating equality checker signals. For each bit position a signal is created and this signal looks whether that specific bits in that specific position are equal or not and it is doing this with XNOR gate. After that, using AND gates between those signals, equality option is evaluated. If this is true, the specified bit position becomes high. Then, AND'ing the more

significant bit equality checkers and the current bits' input value bits less or greater options are checked. This operation made for AgtB and since AltB is possible if only inputs are not equal and not AltB, the two previous operations are NOR'ed. Lastly, a bitwise XOR is implemented by XOR'ing two input vectors.

Results

The schematics and waveform appears as expected. Schematic is slightly complex but it is so because there are many connections and operations are used. However, overall, it fulfills expectations and works properly. There are 3 selection bit inputs and 2 6-bit vector inputs of the multiplexer. Similarly, it has a 6-bit vector output. Depending on the selection data, selected operation connects to output and the result is transferred. The inner components' structures are given in the appendix part.

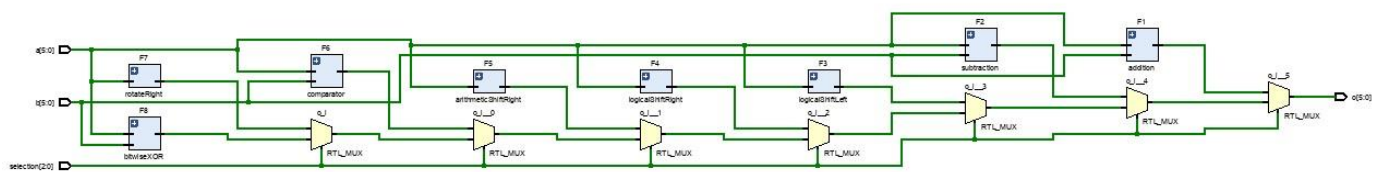


Figure 1: Elaborated Design

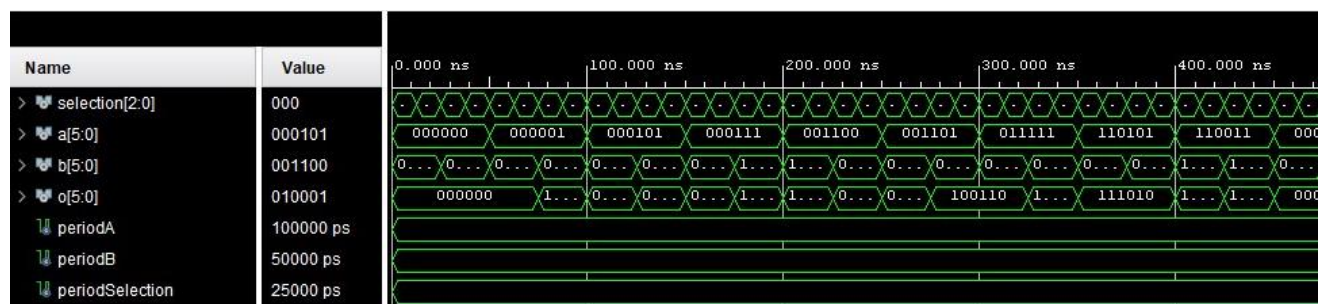


Figure 2: Waveform Graph

Conclusion

The purpose of this experiment was to design an Arithmetic Logic Unit (ALU) which is a combinational circuit that can perform some specific operations. In the lab, a multiplexer is used to build the ALU. 8 distinct function operators are designed and they are used in an ordered hierarchy. The input vectors are assigned to switches (each bit to each switch) and the selection bits are also assigned to switches separately. The bits of the output vector is assigned to LED's on the BASYS 3. During the process, there were minor algorithmic errors but since they were not syntactic, the program worked and bitstream could be generated; however, since it was not matching the expectations, the problem had to be solved. Debugging and correct the mistakes of the working program was hard because there were no error messages. Also since the program consists of distinct and slightly complex component, building the upper parts and dealing with errors were the major problems faces during this lab. Lastly, there is a problem encountered when addition and subtraction are using same halfAdder and fullAdder. When another halfAdder and fullAdder named as halfAdderForSubtraction and halfAdderForSubtraction are created and subtraction function uses this, problem solved. This might be caused an error because the operations are parallel, namely, they are made at the same time. Therefore, two functions are using same component at the same time with variables named same, caused a "multi-driven net" error.

Appendices

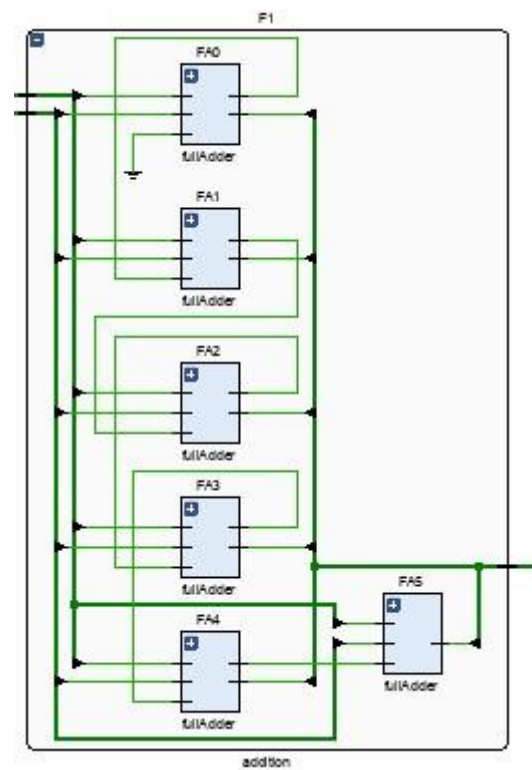


Figure 4: RTL Design of Addition Function

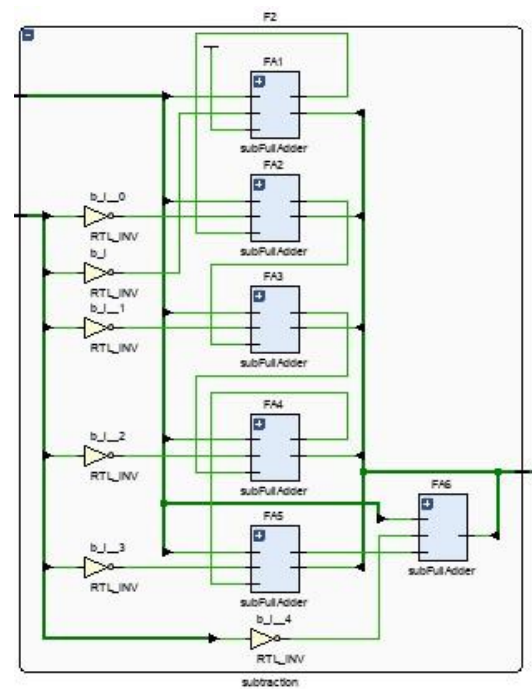


Figure 3: RTL Design of Subtraction Function

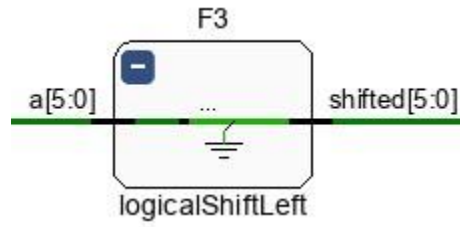


Figure 5: RTL Design of Logical Shift Left

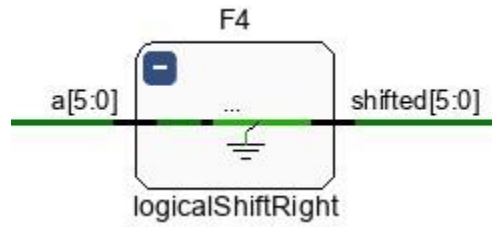


Figure 6: RTL Design of Logical Shift Right

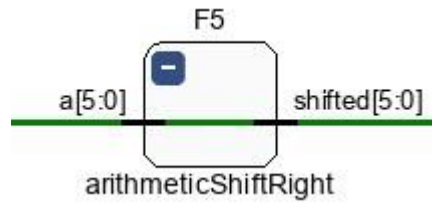


Figure 7: RTL Design of Arithmetic Shift Right

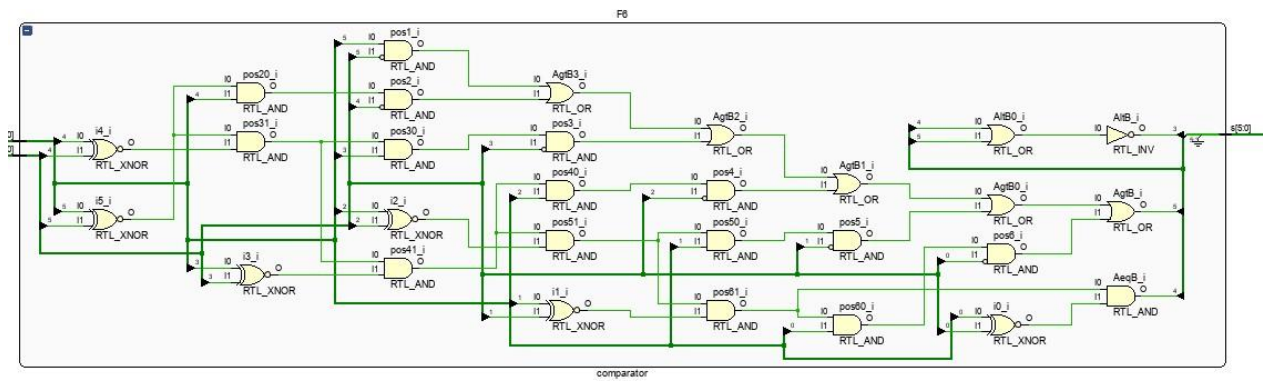


Figure 8: RTL Design of Comparator

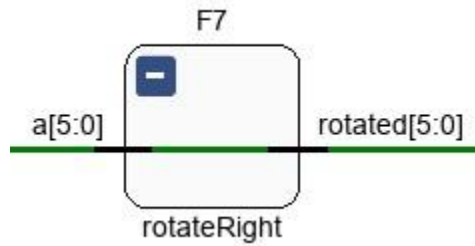


Figure 9: RTL Design of Rotate Right

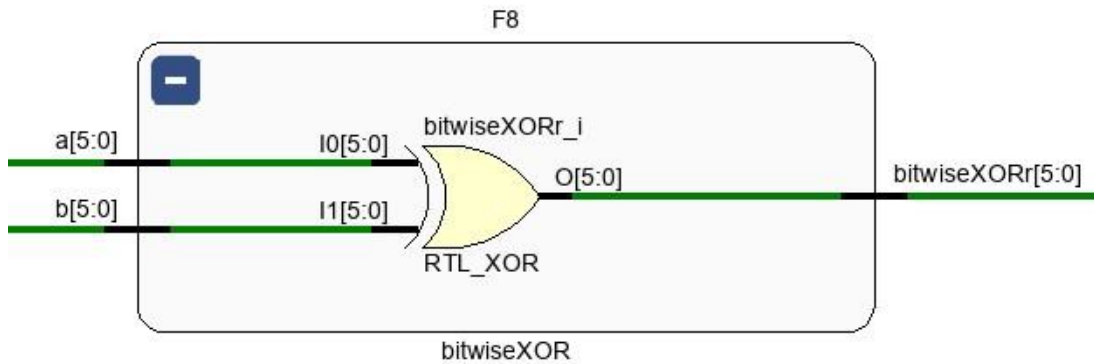


Figure 10: RTL Design of Bitwise XOR

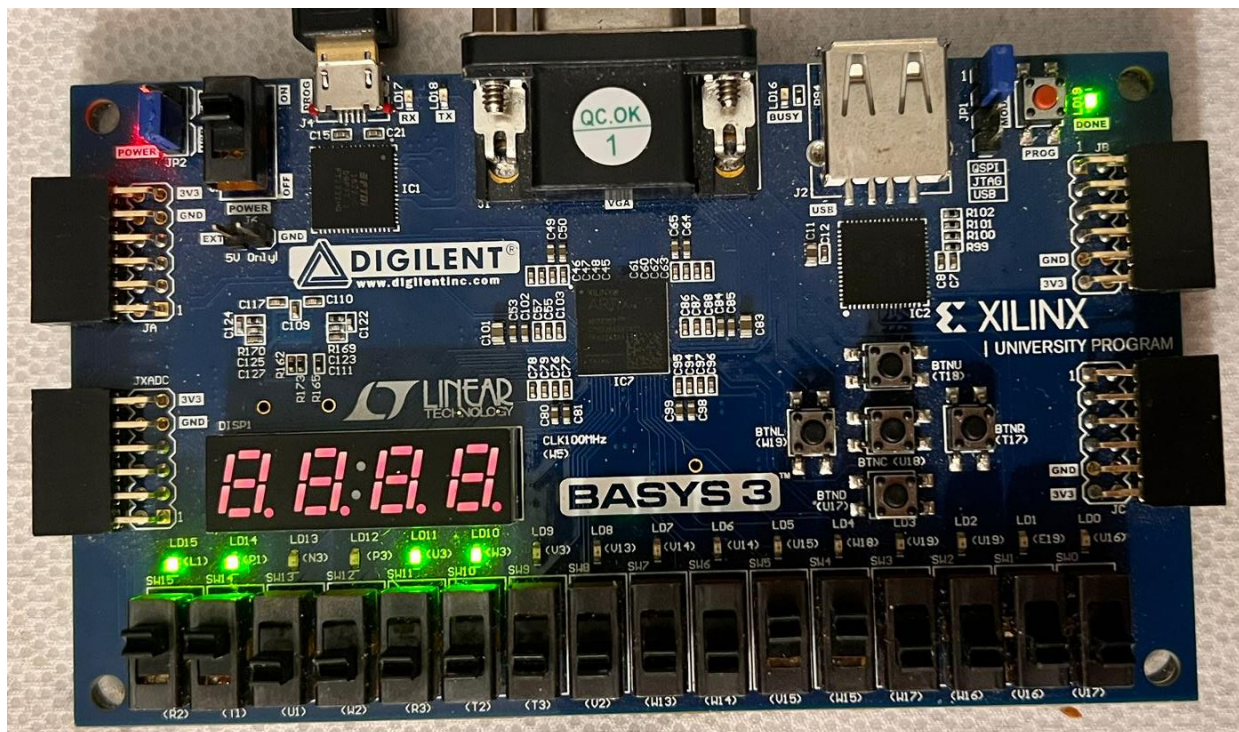


Figure 11: selection = "000", Addition function is selected. Output = "110011"

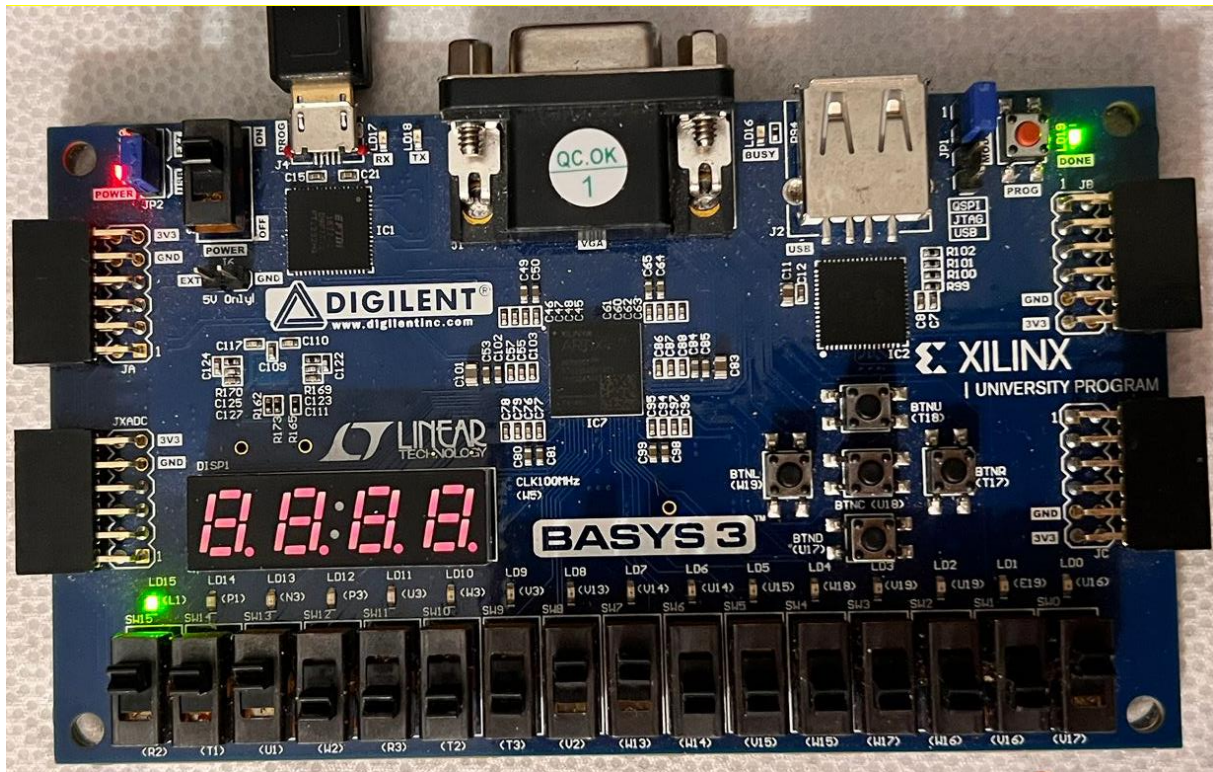


Figure 11: selection = "001", Subtraction function is selected. Output = "10000"

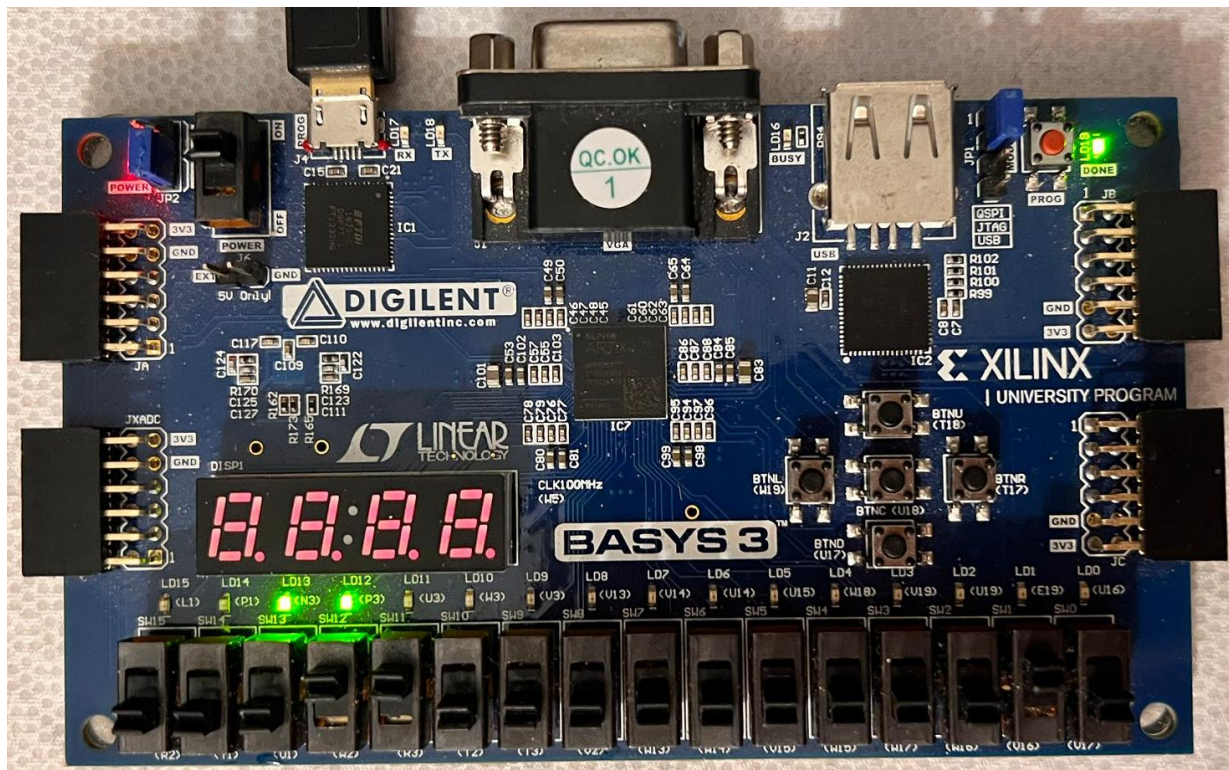


Figure 10: selection = "010", Logical Shift Left is selected. Output = "001100"

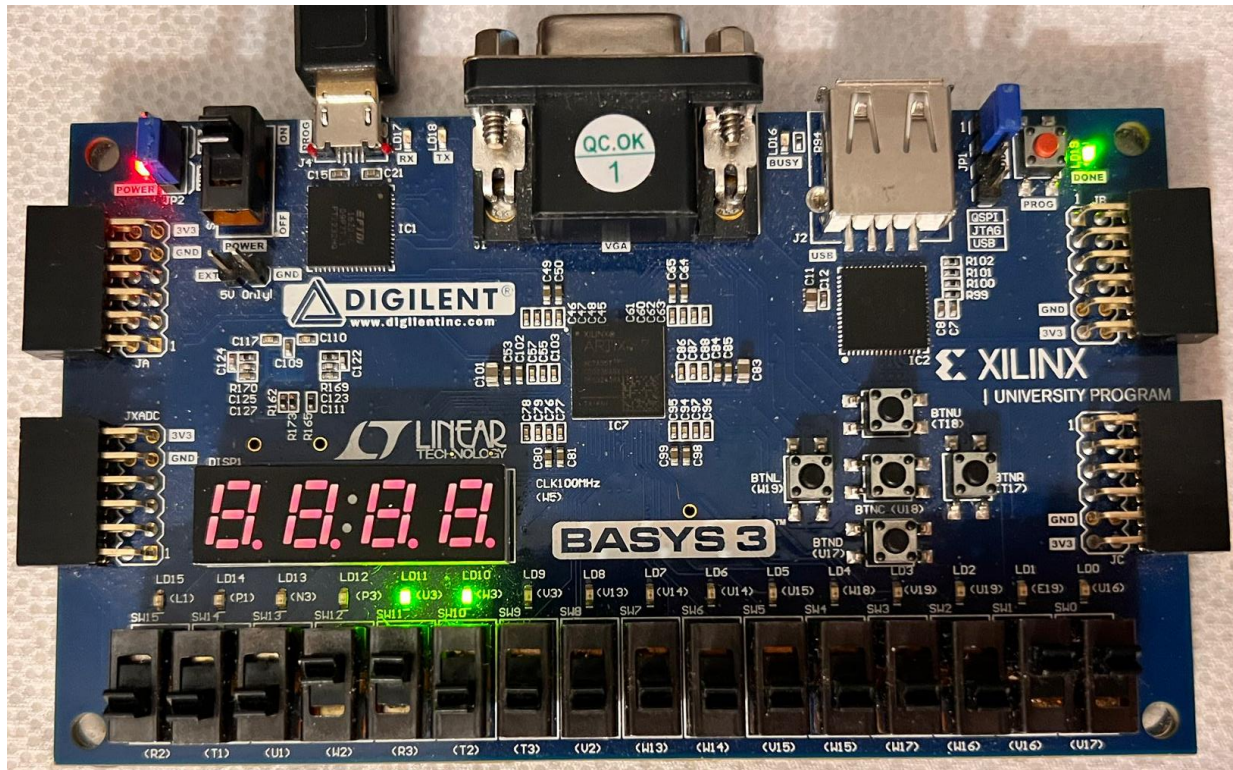


Figure 14: selection = "011", Logical Shift Right is selected. Output = "000011"

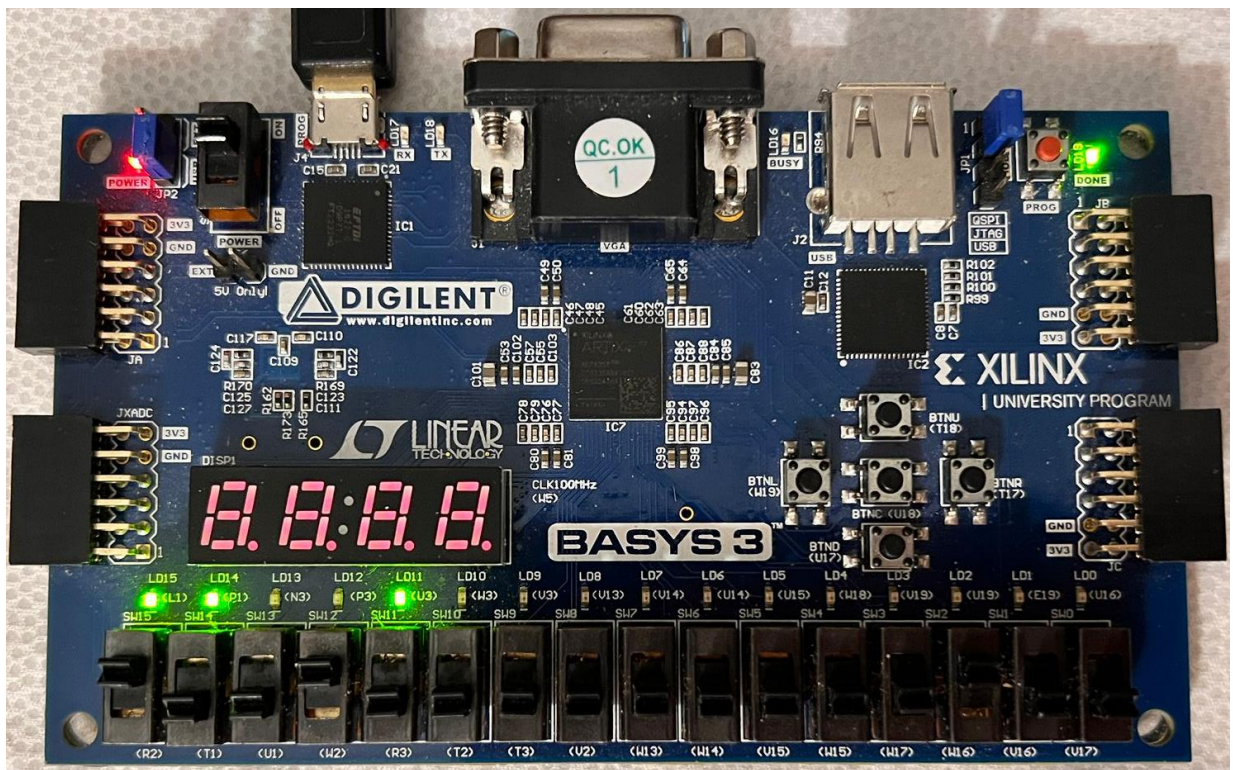


Figure 15: selection = "100", Arithmetic Shift Right is selected. Output = "110010"

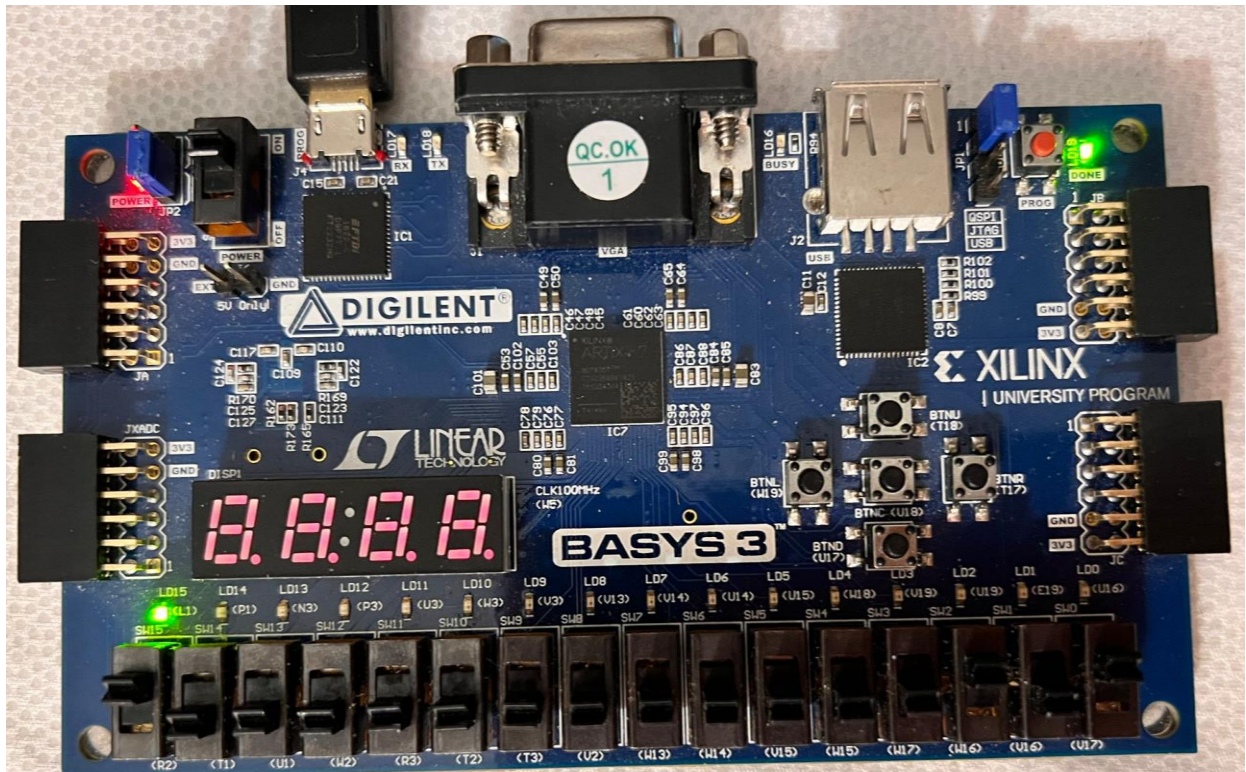


Figure 16: selection = "101", Comparator function is selected. Output = "100000"

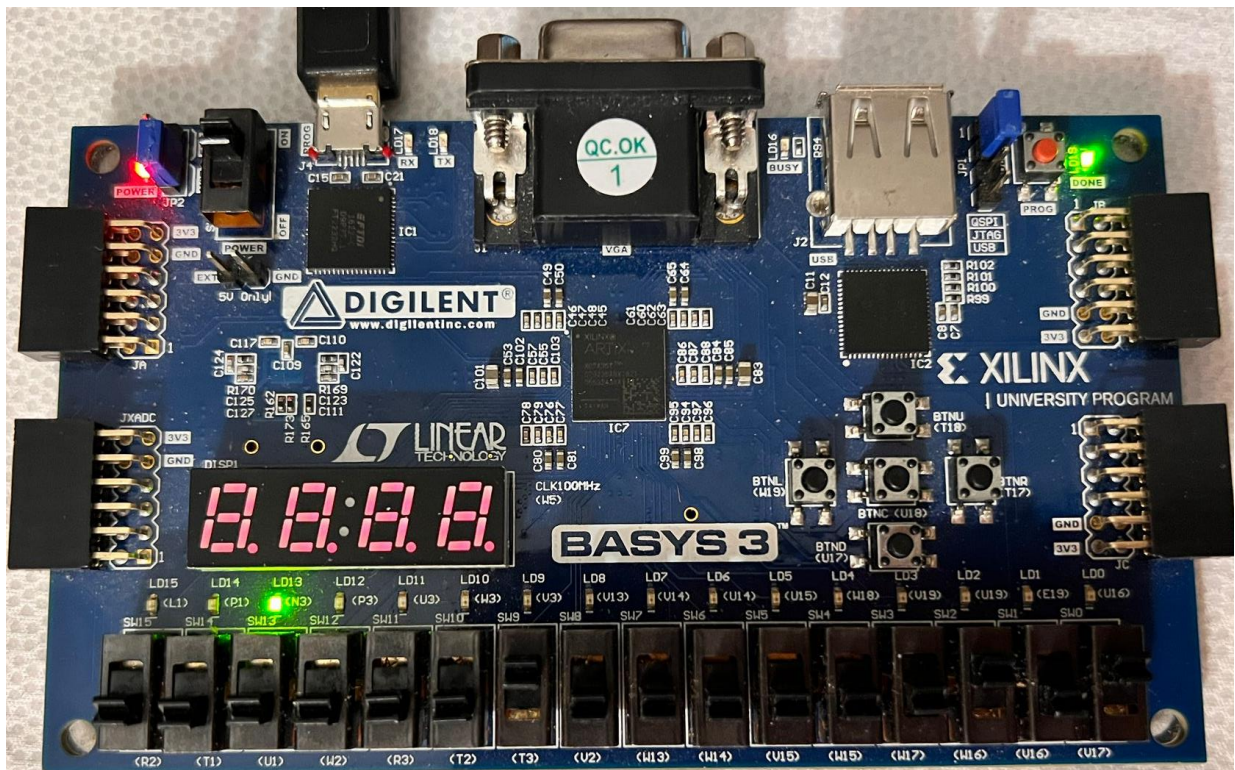


Figure 17: selection = "101", Comparator function is selected. Output = "0010000"

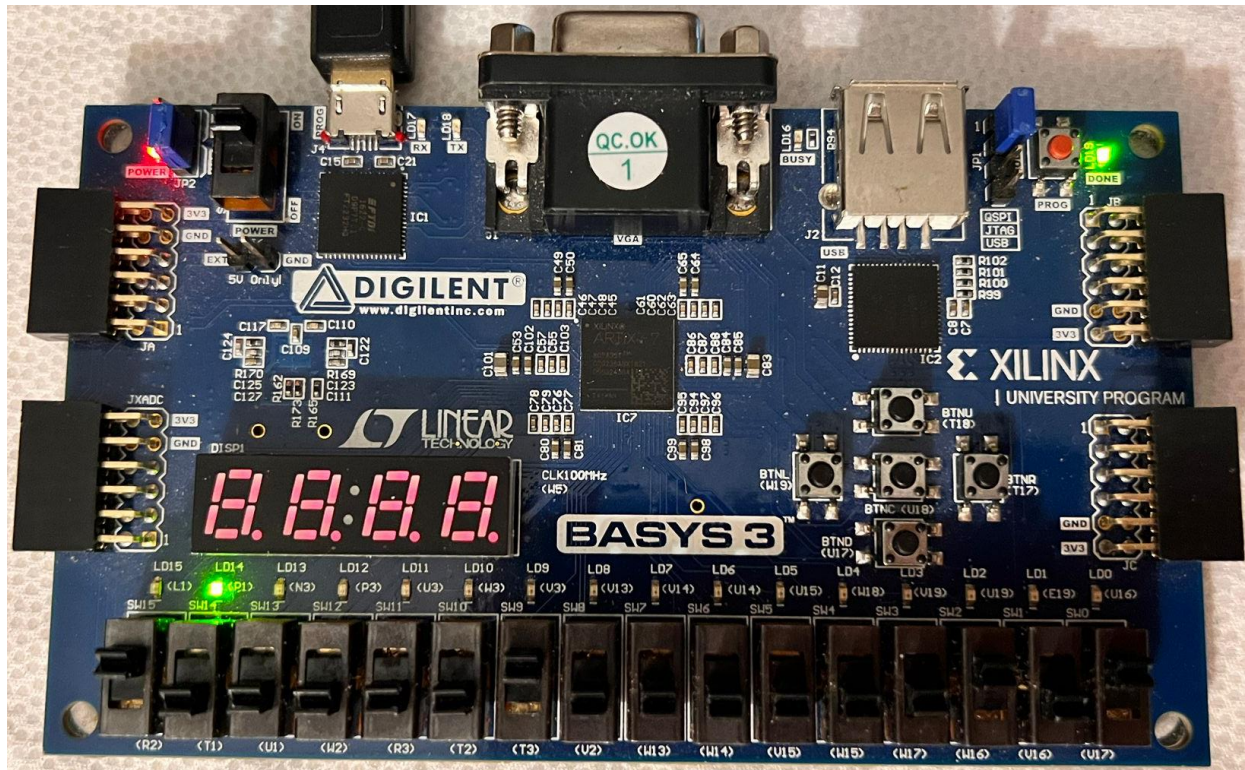


Figure 18: selection = "101", Comparator function is selected. Output = "0100000"

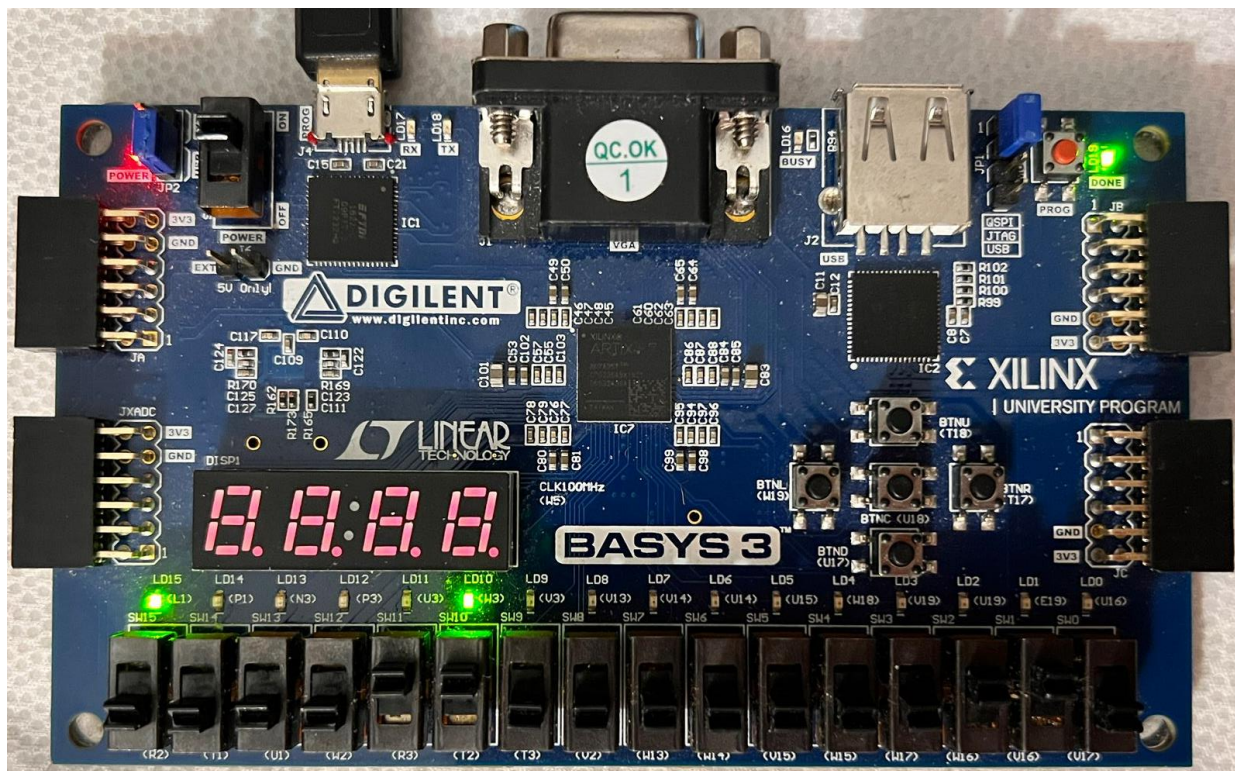


Figure 19: selection = "110", Rotate right is selected. Output = "100001"

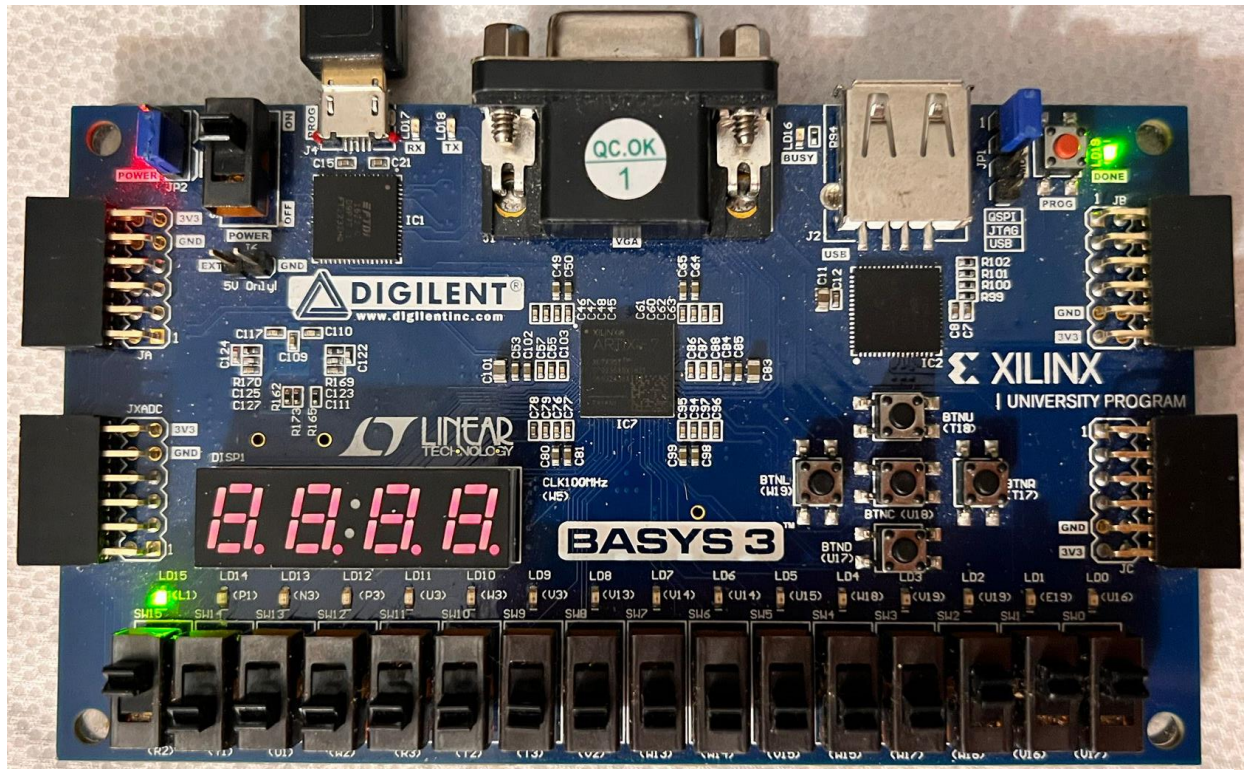


Figure 20: selection = "111", Bitwise XOR function is selected. Output = "100000"

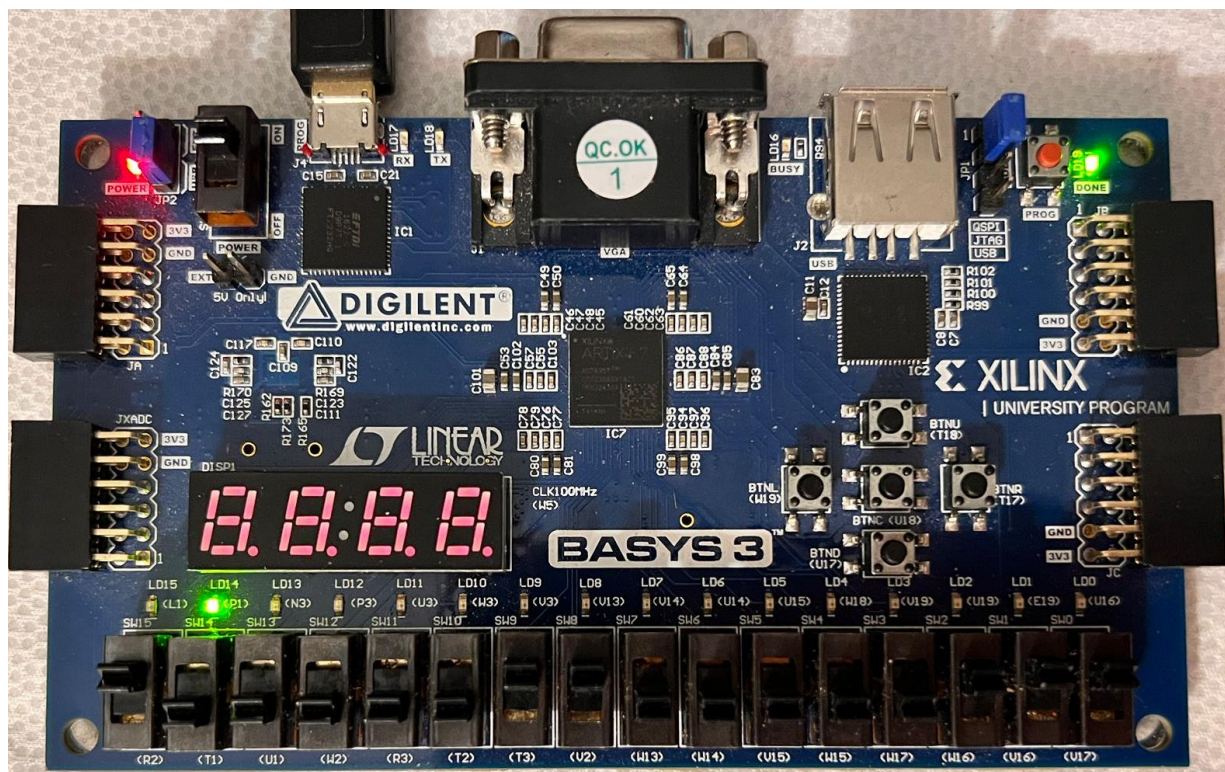


Figure 21: selection = "111", Bitwise XOR function is selected. Output = "010000"


```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

entity muxFile is

```
Port (
    selection : in std_logic_vector (2 downto 0);
    a, b      : in std_logic_vector(5 downto 0);
    o        : out std_logic_vector(5 downto 0));
end muxFile;
```

architecture Behavioral of muxFile is

-- OUTPUTS

```
Signal raddition      : std_logic_vector(5 downto 0);
Signal rsubtraction   : std_logic_vector(5 downto 0);
Signal rarithmeticShiftRight : std_logic_vector(5 downto 0);
Signal rlogicalShiftLeft  : std_logic_vector(5 downto 0);
Signal rlogicalShiftRight : std_logic_vector(5 downto 0);
Signal rcomparator      : std_logic_vector(5 downto 0);
Signal rrotateRight     : std_logic_vector(5 downto 0);
Signal rbitwiseXOR      : std_logic_vector(5 downto 0);
```

component addition is

```
Port (
    a, b: in std_logic_vector (5 downto 0);
    result : out std_logic_vector (5 downto 0));
end component;
```

component subtraction is

```
Port (a, b: in std_logic_vector (5 downto 0);
    difference : out std_logic_vector (5 downto 0));
end component;
```

component logicalShiftLeft is

```
Port (
    a : in std_logic_vector (5 downto 0);
    shifted : out std_logic_vector(5 downto 0));
end component;
```

component logicalShiftRight is

```
Port (
    a : in std_logic_vector (5 downto 0);
    shifted : out std_logic_vector(5 downto 0));
end component;
```

component arithmeticShiftRight is

```
Port (  
    a : in std_logic_vector (5 downto 0);  
    shifted : out std_logic_vector(5 downto 0));  
end component;
```

component comparator is

```
Port (  
    a, b : in std_logic_vector (5 downto 0);  
    s : out std_logic_vector (5 downto 0));  
end component;
```

component rotateRight is

```
Port (  
    a : in std_logic_vector (5 downto 0);  
    rotated : out std_logic_vector (5 downto 0));  
end component;
```

component bitwiseXOR is

```
Port (  
    a, b : in std_logic_vector(5 downto 0);  
    bitwiseXORr : out std_logic_vector (5 downto 0));  
end component;  
begin
```

F1: addition port map(a, b, raddition);

F2: subtraction port map(a, b, rsubtraction);

F3: logicalShiftLeft port map(a, rlogicalShiftLeft);

F4: logicalShiftRight port map(a, rlogicalShiftRight);

F5: arithmeticShiftRight port map(a, rarithmeticShiftRight);

F6: comparator port map(a, b, rcomparator);

F7: rotateRight port map(a, rrotateRight);

F8: bitwiseXOR port map(a, b, rbitwiseXOR);

process(raddition, rsubtraction, rarithmeticShiftRight, rlogicalShiftLeft, rlogicalShiftRight,
rcomparator, rrotateRight, rbitwiseXOR) begin

if selection = "000" then

o <= raddition;

elsif selection = "001" then

o <= rsubtraction;

elsif selection = "010" then

o <= rlogicalShiftLeft;

elsif selection = "011" then


```

        o <= rlogicalShiftRight;
    elsif selection = "100" then
        o <= rarithmeticShiftRight;
    elsif selection = "101" then
        o <= rcomparator;
    elsif selection = "110" then
        o <= rrotateRight;
    else
        o <= rbitwiseXOR;
    end if;

end process;

end Behavioral;

```

Code 1: Main ALU Code

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity halfAdder is
    Port (
        a, b : in std_logic;
        sum, carry : out std_logic );
end halfAdder;

architecture Behavioral of halfAdder is

begin
    sum <= a xor b;
    carry <= a and b;

end Behavioral;

```

Code 2: Half Adder Code for Addition

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity fullAdder is
    Port (
        a, b, cin : in std_logic;
        s, cout : out std_logic);
end fullAdder;

```

architecture Behavioral of fullAdder is

```
signal c0, c1, s0, s1 : std_logic;
```

```
component halfAdder is
```

```
Port(
```

```
    a, b : in std_logic;
```

```
    sum, carry : out std_logic);
```

```
end component;
```

```
begin
```

```
HA1: halfAdder port map(a, b, s0, c0);
```

```
HA2: halfAdder port map(cin, s0, s1, c1);
```

```
s <= s1;
```

```
cout <= c0 or c1;
```

```
end Behavioral;
```

Code 3: Full Adder Code for Addition

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity addition is
```

```
Port (
```

```
    a: in std_logic_vector (5 downto 0);
```

```
    b: in std_logic_vector (5 downto 0);
```

```
    result : out std_logic_vector (5 downto 0));
```

```
end addition;
```

architecture Behavioral of addition is

```
component fullAdder is
```

```
Port (
```

```
    a, b, cin : in std_logic;
```

```
    s, cout : out std_logic);
```

```
end component;
```

```
signal cin: std_logic := '0';
```

```
signal c0, c1, c2, c3, c4, c5 : std_logic;
```

```
signal s0, s1, s2, s3, s4, s5 : std_logic;
```

```
begin
```

```

FA0: fullAdder port map (a(0), b(0), cin, s0, c0);
FA1: fullAdder port map (a(1), b(1), c0, s1, c1);
FA2: fullAdder port map (a(2), b(2), c1, s2, c2);
FA3: fullAdder port map (a(3), b(3), c2, s3, c3);
FA4: fullAdder port map (a(4), b(4), c3, s4, c4);
FA5: fullAdder port map (a(5), b(5), c4, s5, c5);

```

```

result(0) <= s0;
result(1) <= s1;
result(2) <= s2;
result(3) <= s3;
result(4) <= s4;
result(5) <= s5;

```

```
end Behavioral;
```

Code 6: Addition Code

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

```

entity subHalfAdder is
    Port (
        a, b : in std_logic;
        sum, carry : out std_logic );
end subHalfAdder;

```

```
architecture Behavioral of subHalfAdder is
```

```

begin
    sum <= a xor b;
    carry <= a and b;

```

```
end Behavioral;
```

Code 7: Half Adder Code for Subtraction

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

```

entity subFullAdder is
    Port (
        a, b, cin : in std_logic;
        s, cout : out std_logic);

```



```
end subFullAdder;
```

```
architecture Behavioral of subFullAdder is
```

```
signal c0, c1, s0, s1 : std_logic;
```

```
    component subHalfAdder is
```

```
        Port(
```

```
            a, b : in std_logic;
```

```
            sum, carry : out std_logic);
```

```
    end component;
```

```
begin
```

```
    HA1: subHalfAdder port map(a, b, s0, c0);
```

```
    HA2: subHalfAdder port map(cin, s0, s1, c1);
```

```
    s <= s1;
```

```
    cout <= c0 or c1;
```

```
end Behavioral;
```

Code 8: Full Adder Code for Subtraction

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity subtraction is
```

```
    Port (a, b: in std_logic_vector (5 downto 0);
```

```
          difference : out std_logic_vector (5 downto 0));
```

```
end subtraction;
```

```
architecture Behavioral of subtraction is
```

```
    component subFullAdder is
```

```
        Port (
```

```
            a, b, cin : in std_logic;
```

```
            s, cout : out std_logic);
```

```
    end component;
```

```
signal c0, c1, c2, c3, c4, c5 : std_logic;
```

```
signal s0, s1, s2, s3, s4, s5 : std_logic;
```

```
begin
```

```
    FA1: subFullAdder port map (a(0), not(b(0)), '1', s0, c0);
```

```

FA2: subFullAdder port map (a(1), not(b(1)), c0, s1, c1);
FA3: subFullAdder port map (a(2), not(b(2)), c1, s2, c2);
FA4: subFullAdder port map (a(3), not(b(3)), c2, s3, c3);
FA5: subFullAdder port map (a(4), not(b(4)), c3, s4, c4);
FA6: subFullAdder port map (a(5), not(b(5)), c4, s5, c5);

```

```

difference(0) <= s0;
difference(1) <= s1;
difference(2) <= s2;
difference(3) <= s3;
difference(4) <= s4;
difference(5) <= s5;

```

```

end Behavioral;

```

Code 5: Subtraction Code

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

```

entity logicalShiftLeft is
    Port (a : in std_logic_vector (5 downto 0);
          shifted : out std_logic_vector(5 downto 0));
end logicalShiftLeft;

```

```

architecture Behavioral of logicalShiftLeft is
begin
    shifted <= a(4 downto 0) & '0';

```

```

end Behavioral;

```

Code 6: Logical Shift Left Code

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

```

entity logicalShiftRight is
    Port (a : in std_logic_vector (5 downto 0);
          shifted : out std_logic_vector(5 downto 0));
end logicalShiftRight;

```

```
architecture Behavioral of logicalShiftRight is
begin
    shifted <= '0' & a(5 downto 1);
```

```
end Behavioral;
```

Code 7: Logical Shift Right Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity arithmeticShiftRight is
    Port (a : in std_logic_vector (5 downto 0);
          shifted : out std_logic_vector(5 downto 0));
end arithmeticShiftRight;
```

```
architecture Behavioral of arithmeticShiftRight is
begin
    shifted <= a(5) & a(5 downto 1);
```

```
end Behavioral;
```

Code 8: Arithmetic Shift Right Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity comparator is
    Port (
        a, b: in std_logic_vector (5 downto 0);
        s : out std_logic_vector (5 downto 0));
end comparator;
```

```
architecture Behavioral of comparator is
```

```
    signal i0 : std_logic := a(0) xnor b(0);
    signal i1 : std_logic := a(1) xnor b(1);
    signal i2 : std_logic := a(2) xnor b(2);
    signal i3 : std_logic := a(3) xnor b(3);
    signal i4 : std_logic := a(4) xnor b(4);
    signal i5 : std_logic := a(5) xnor b(5);
```



```
signal AeqB : std_logic;  
signal AgtB : std_logic;  
signal AltB : std_logic;
```

```
signal pos1 : std_logic;  
signal pos2 : std_logic;  
signal pos3 : std_logic;  
signal pos4 : std_logic;  
signal pos5 : std_logic;  
signal pos6 : std_logic;
```

```
begin
```

```
    AeqB <= i5 and i4 and i3 and i2 and i1 and i0;
```

```
    pos1 <= a(5) and not b(5);  
    pos2 <= i5 and a(4) and (not b(4));  
    pos3 <= i5 and i4 and a(3) and (not b(3));  
    pos4 <= i5 and i4 and i3 and a(2) and (not b(2));  
    pos5 <= i5 and i4 and i3 and i2 and a(1) and (not b(1));  
    pos6 <= i5 and i4 and i3 and i2 and i1 and a(0) and (not b(0));
```

```
    AgtB <= (pos1) or (pos2) or (pos3) or (pos4) or (pos5) or (pos6);  
    AltB <= not (AeqB or AgtB);
```

```
    s(0) <= '0';  
    s(1) <= '0';  
    s(2) <= '0';  
    s(3) <= AltB;  
    s(4) <= AeqB;  
    s(5) <= AgtB;
```

```
end Behavioral;
```

Code 9: Comparator Code

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity rotateRight is
```

```
    Port ( a : in std_logic_vector (5 downto 0);  
          rotated : out std_logic_vector (5 downto 0));
```

```
end rotateRight;
```

architecture Behavioral of rotateRight is

begin

rotated <= a(0) & a(5 downto 1);

end Behavioral;

Code 10: Rotate Right Code

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

entity bitwiseXOR is

Port (a, b: in std_logic_vector (5 downto 0);

result: out std_logic_vector (5 downto 0));

end bitwiseXOR;

architecture Behavioral of bitwiseXOR is

begin

result <= a xor b;

end Behavioral;

Code 11: Bitwise XOR Code

Switches

set_property PACKAGE_PIN V17 [get_ports selection[0]]

set_property IOSTANDARD LVCMOS33 [get_ports selection[0]]

set_property PACKAGE_PIN V16 [get_ports selection[1]]

set_property IOSTANDARD LVCMOS33 [get_ports selection[1]]

set_property PACKAGE_PIN W16 [get_ports selection[2]]

set_property IOSTANDARD LVCMOS33 [get_ports selection[2]]

##set_property PACKAGE_PIN W17 [get_ports {sw[3]}]

set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]

set_property PACKAGE_PIN W15 [get_ports b[0]]

set_property IOSTANDARD LVCMOS33 [get_ports b[0]]

set_property PACKAGE_PIN V15 [get_ports b[1]]

set_property IOSTANDARD LVCMOS33 [get_ports b[1]]

set_property PACKAGE_PIN W14 [get_ports b[2]]

set_property IOSTANDARD LVCMOS33 [get_ports b[2]]

set_property PACKAGE_PIN W13 [get_ports b[3]]

set_property IOSTANDARD LVCMOS33 [get_ports b[3]]

set_property PACKAGE_PIN V2 [get_ports b[4]]

set_property IOSTANDARD LVCMOS33 [get_ports b[4]]

```
set_property PACKAGE_PIN T3 [get_ports b[5]]
    set_property IOSTANDARD LVCMOS33 [get_ports b[5]]
set_property PACKAGE_PIN T2 [get_ports a[0]]
    set_property IOSTANDARD LVCMOS33 [get_ports a[0]]
set_property PACKAGE_PIN R3 [get_ports a[1]]
    set_property IOSTANDARD LVCMOS33 [get_ports a[1]]
set_property PACKAGE_PIN W2 [get_ports a[2]]
    set_property IOSTANDARD LVCMOS33 [get_ports a[2]]
set_property PACKAGE_PIN U1 [get_ports a[3]]
    set_property IOSTANDARD LVCMOS33 [get_ports a[3]]
set_property PACKAGE_PIN T1 [get_ports a[4]]
    set_property IOSTANDARD LVCMOS33 [get_ports a[4]]
set_property PACKAGE_PIN R2 [get_ports a[5]]
    set_property IOSTANDARD LVCMOS33 [get_ports a[5]]
```

LEDs

```
#set_property PACKAGE_PIN U16 [get_ports {LED[0]}]
#    set_property IOSTANDARD LVCMOS33 [get_ports {LED[0]}]
#set_property PACKAGE_PIN E19 [get_ports {LED[1]}]
#    set_property IOSTANDARD LVCMOS33 [get_ports {LED[1]}]
#set_property PACKAGE_PIN U19 [get_ports {LED[2]}]
#    set_property IOSTANDARD LVCMOS33 [get_ports {LED[2]}]
#set_property PACKAGE_PIN V19 [get_ports {LED[3]}]
#    set_property IOSTANDARD LVCMOS33 [get_ports {LED[3]}]
#set_property PACKAGE_PIN W18 [get_ports {LED[4]}]
#    set_property IOSTANDARD LVCMOS33 [get_ports {LED[4]}]
#set_property PACKAGE_PIN U15 [get_ports {LED[5]}]
#    set_property IOSTANDARD LVCMOS33 [get_ports {LED[5]}]
#set_property PACKAGE_PIN U14 [get_ports {LED[6]}]
#    set_property IOSTANDARD LVCMOS33 [get_ports {LED[6]}]
#set_property PACKAGE_PIN V14 [get_ports {LED[7]}]
#    set_property IOSTANDARD LVCMOS33 [get_ports {LED[7]}]
#set_property PACKAGE_PIN V13 [get_ports {LED[8]}]
#    set_property IOSTANDARD LVCMOS33 [get_ports {LED[8]}]
#set_property PACKAGE_PIN V3 [get_ports {LED[9]}]
#    set_property IOSTANDARD LVCMOS33 [get_ports {LED[9]}]
set_property PACKAGE_PIN W3 [get_ports o[0]]
    set_property IOSTANDARD LVCMOS33 [get_ports o[0]]
set_property PACKAGE_PIN U3 [get_ports o[1]]
    set_property IOSTANDARD LVCMOS33 [get_ports o[1]]
set_property PACKAGE_PIN P3 [get_ports o[2]]
    set_property IOSTANDARD LVCMOS33 [get_ports o[2]]
set_property PACKAGE_PIN N3 [get_ports o[3]]
```

```

        set_property IOSTANDARD LVCMOS33 [get_ports o[3]]
set_property PACKAGE_PIN P1 [get_ports o[4]]
        set_property IOSTANDARD LVCMOS33 [get_ports o[4]]
set_property PACKAGE_PIN L1 [get_ports o[5]]
        set_property IOSTANDARD LVCMOS33 [get_ports o[5]]

```

Code 12: Constraint Code

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

```

entity testBench is
    -- Port ();
end testBench;

```

architecture Behavioral of testBench is

```

    component muxFile is
    Port (
        selection : in std_logic_vector (2 downto 0);
        a, b      : in std_logic_vector(5 downto 0);
        o         : out std_logic_vector(5 downto 0));
    end component;

```

```

    signal selection : std_logic_vector (2 downto 0);
    signal a, b      : std_logic_vector(5 downto 0);
    signal o         : std_logic_vector(5 downto 0);

```

```

    constant periodA : time := 100ns;
    constant periodB : time := 50ns;
    constant periodSelection : time := 25ns;

```

```

begin
    UUT: muxFile port map(
        selection => selection,
        a => a,
        b => b,
        o => o);

```

```

    processOfA : process begin

```



```
a <= "000000";
wait for periodA/2;
a <= "000001";
wait for periodA/2;
a <= "000101";
wait for periodA/2;
a <= "000111";
wait for periodA/2;
a <= "001100";
wait for periodA/2;
a <= "001101";
wait for periodA/2;
a <= "011111";
wait for periodA/2;
a <= "110101";
wait for periodA/2;
a <= "110011";
wait for periodA/2;
a <= "111111";
end process;
```

processOfB : process begin

```
b <= "000000";
wait for periodB/2;
b <= "000001";
wait for periodB/2;
b <= "000101";
wait for periodB/2;
b <= "000111";
wait for periodB/2;
b <= "001100";
wait for periodB/2;
b <= "001101";
wait for periodB/2;
b <= "011111";
wait for periodB/2;
b <= "110101";
wait for periodB/2;
b <= "110011";
wait for periodB/2;
b <= "111111";
end process;
```

processSelection: process begin

```
selection <= "000";  
wait for periodSelection/2;  
selection <= "001";  
wait for periodSelection/2;  
selection <= "010";  
wait for periodSelection/2;  
selection <= "011";  
wait for periodSelection/2;  
selection <= "100";  
wait for periodSelection/2;  
selection <= "101";  
wait for periodSelection/2;  
selection <= "110";  
wait for periodSelection/2;  
selection <= "111";  
wait for periodSelection/2;  
end process;
```

```
stim_proc: process begin  
  -- ALU at reset  
  wait for 500 ns;  
  wait for periodA*10;  
  wait;  
end process;
```

end Behavioral;

Code 13: Test Bench Code

References

<https://www.techtarget.com/whatis/definition/arithmetic-logic-unit-ALU>

https://en.wikipedia.org/wiki/Arithmetic_logic_unit